HDOC Day-61

Two Sum Using Pointers and Nested Loops in C

Problem Statement

Read the size and elements of a variable-length array of positive integers and an integer target. Pass the array by address to two_sum(). Using nested loops, find two distinct zero-based indices whose elements sum to the target. Display the indices separated by a space, or -1 -1 when no pair exists. Assume at most one valid pair.

Algorithm

1. Start.
2. Read size and ensure it is a positive integer (maximum 10000 in this implementation).
3. Declare a variable-length integer array nums[size].
4. Read all array elements; reject any element that is not positive.
5. Read the integer target.
6. Call two_sum(nums, size, target), passing the array address.
7. In two_sum(), use outer loop i from 0 to size-2 and inner loop j from i+1 to size-1.
8. If nums[i]+nums[j] equals target, display i and j and return immediately.
9. If the loops finish without a match, display -1 -1.
10. Stop.

Test Case Table

All indices are zero-based. For each test case, enter the array size, array elements, and target in that order unless the input is rejected earlier.

Test case	Size	Array elements	Target	Expected result
1	4	2 7 11 15	9	0 1
2	3	3 2 4	6	1 2
3	2	3 3	6	0 1
4	3	1 2 3	7	-1 -1
5	1	5	10	-1 -1
6	4	1 4 6 9	10	0 3
7	3	2 5 8	13	1 2
8	3	1 2 3	0	-1 -1
9	0	(not read)	(not read)	Invalid size
10	3	2 -1 4	(not read)	Invalid element

C Program

```
#include <stdio.h>

void two_sum(const int *nums, int size, int target)
{
    int i, j;
    for (i = 0; i < size - 1; i++)
    {
        for (j = i + 1; j < size; j++)
```

```
        {
            if ((long long)*(nums + i) + *(nums + j) == target)
            {
                printf("%d %d\n", i, j);
                return;
            }
        }
    }
    printf("-1 -1\n");
}

int main(void)
{
    int size, i, target;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000)
    {
        printf("Invalid size. Enter a positive integer up to 10000.\n");
        return 1;
    }

    int nums[size];
    printf("Enter %d positive array elements:\n", size);
    for (i = 0; i < size; i++)
    {
        if (scanf("%d", nums + i) != 1 || *(nums + i) <= 0)
        {
            printf("Invalid element. Enter positive integers only.\n");
            return 1;
        }
    }

    printf("Enter target: ");
    if (scanf("%d", &target) != 1)
    {
        printf("Invalid target.\n");
        return 1;
    }
    printf("Indices: ");
    two_sum(nums, size, target);
    return 0;
}
```

Compilation and Execution Commands

```
gcc hdoc61.c -o hdoc61
./hdoc61
```

Screenshots

Compilation and Execution

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc61.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc61.c -o hdoc61
```

Test Case 1

Input: size = 4; array = 2 7 11 15; target = 9. Expected: 0 1. First example; 2+7=9.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 4
Enter 4 positive array elements:
2 7 11 15
Enter target: 9
Indices: 0 1
```

Test Case 2

Input: size = 3; array = 3 2 4; target = 6. Expected: 1 2. Second example; 2+4=6.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 3
Enter 3 positive array elements:
3 2 4
Enter target: 6
Indices: 1 2
```

Test Case 3

Input: size = 2; array = 3 3; target = 6. Expected: 0 1. Equal values at distinct indices.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 2
Enter 2 positive array elements:
3 3
Enter target: 6
Indices: 0 1
```

Test Case 4

Input: size = 3; array = 1 2 3; target = 7. Expected: -1 -1. No matching pair.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 3
Enter 3 positive array elements:
1 2 3
Enter target: 7
Indices: -1 -1
```

Test Case 5

Input: size = 1; array = 5; target = 10. Expected: -1 -1. Only one element; no distinct pair.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 1
Enter 1 positive array elements:
5
Enter target: 10
Indices: -1 -1
```

Test Case 6

Input: size = 4; array = 1 4 6 9; target = 10. Expected: 0 3. First and last elements sum to target.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 4
Enter 4 positive array elements:
1 4 6 9
Enter target: 10
Indices: 0 3
```

Test Case 7

Input: size = 3; array = 2 5 8; target = 13. Expected: 1 2. Matching pair at later indices.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 3
Enter 3 positive array elements:
2 5 8
Enter target: 13
Indices: 1 2
```

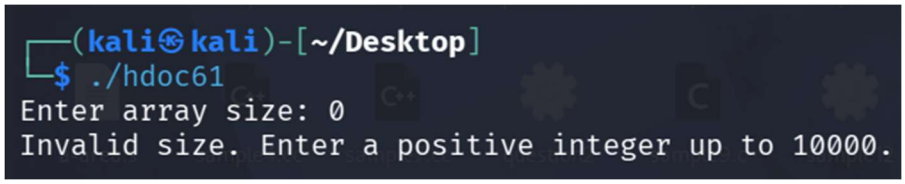
Test Case 8

Input: size = 3; array = 1 2 3; target = 0. Expected: -1 -1. No positive pair can sum to zero.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc61
Enter array size: 3
Enter 3 positive array elements:
1 2 3
Enter target: 0
Indices: -1 -1
```

Test Case 9

Input: size = 0; array = (not read); target = (not read). Expected: Invalid size. Size must be positive.



Test Case 10

Input: size = 3; array = 2 -1 4; target = (not read). Expected: Invalid element. Reject non-positive element.

